Tenis Ahora — Informe final de la API de prueba

Rust + axum + SQLite · carpeta **D:\fede\api** · versión 1.0.0 · 23 de octubre de 2026

Este informe documenta la API de prueba construida para que el frontend **Tenis-Ahora-Frontend** opere contra datos persistidos, de modo que el TP pueda probarse de forma realista. Está dirigido a quien construya la **versión definitiva** de la API: describe qué se hizo, en qué versiones, cómo está armado, qué contrato HTTP espera el front y qué conviene cambiar al pasar a una implementación de producción.

Aspecto	Detalle
Propósito	Backend de prueba para el front. No está pensado para tráfico real ni para producción.
Stack	Rust 2021 · tokio · axum 0.8 · tower-http · rusqlite (SQLite 3.50.4 por DLL oficial) · bcrypt · JWT HS256.
Persistencia	Un archivo SQLite (data/tenis_ahora.sqlite) con una tabla documents (colección, id, JSON).
Datos iniciales	seed/seed.json (exportado del mock del front). Contraseña 123456 para todos los usuarios.
Contrato	3 rutas públicas, CRUD genérico para 15 colecciones y 29 endpoints de negocio bajo /api. Errores { "error": "..." }.
Front	src/services/httpApi.js + withBackend(): usa la API cuando /api/health responde y la sesión no es Modo Demo.
Pruebas	tests/smoke.py: 45/45 verificaciones OK · recorrido del front en navegador contra la API.
Despliegue	start-servidor.ps1 compila API y front y deja la API sirviendo ambos en el puerto 5090, accesible por nip.io.

1. Historial de versiones

Cada versión cierra un grupo de cambios; el detalle completo está en **api/CHANGELOG.md**.

Versión	Contenido	Estado
0.1.0	Proyecto axum + tokio. Store sobre SQLite (tabla documents, WAL) enlazando la DLL oficial de SQLite sin compilador C. Seed automático. Autenticación: login, registro, /auth/yo, JWT HS256 de 8 h, middleware Bearer. /health. Errores uniformes.	Cerrada
0.2.0	CRUD genérico para las 15 colecciones con ids con los mismos prefijos que el front y las reglas de alta, modificación y baja del mock (unicidad, cupos, recálculos, cascadas). passwordHash nunca se expone.	Cerrada
0.3.0	Endpoints de negocio: canchas (estado, disponibilidad), stock (ajuste), reservas (alta con recibo de seña, pago de saldo, cancelación con reembolso o cargo), torneos y ligas (reglamento, inscripciones, enfrentamientos, resultado con tabla), docentes (disponibilidad, certificación), clases (alumnos, asistencia), turnos particulares, facturas, reembolsos, descuentos por usuario, reset.	Cerrada
0.4.0	Integración con el front: httpApi.js, withBackend(), AuthContext con dni y memberNumber. La API sirve el build del front con fallback SPA.	Cerrada
1.0.0	Scripts de arranque y parada, prueba de humo, verificación en navegador, acceso desde otra PC por nip.io, este informe.	Cerrada

2. Arquitectura

2.1 Flujo de una petición

```
Navegador (front React)
   |  fetch /api/...  (Authorization: Bearer <jwt>)
   v
axum Router  --nest /api-->  rutas públicas (health, auth/login, auth/registrar)
   |                         rutas protegidas  --middleware require_auth (verifica HS256, exp)-->
   |                              business.rs   (endpoints con reglas de negocio)
   |                              crud.rs       (GET/POST /{col}, GET/PUT/DELETE /{col}/{id})
   |                                   |
   |                                   v
   |                              db.rs Store  (Mutex<rusqlite::Connection>)
   |                                   |
   |                                   v
   |                              SQLite  documents(collection, id, data JSON, created_at, updated_at)
   |
   +-- fallback_service: ServeDir(../Tenis-Ahora-Frontend/dist) -> index.html (SPA)
CorsLayer::permissive()  +  TraceLayer (logs)
```

2.2 Módulos

Archivo	Responsabilidad
src/main.rs	Configuración por variables de entorno, apertura de la base, seed, construcción del router, archivos estáticos del front, servidor.
src/db.rs	Store: open, list, get, require (404), insert (409 si existe), upsert, delete, clear, counts. Un solo Mutex: suficiente para pruebas.
src/auth.rs	bcrypt (cost 4), firma y verificación JWT HS256 (hmac + sha2 + base64), middleware, login, registro, /auth/yo, AuthResponseDto.
src/crud.rs	Validación de colección, generación de ids, defaults y validaciones de alta y modificación por colección, chequeos de baja, handlers genéricos.
src/business.rs	Endpoints de negocio. Es el equivalente servidor de la lógica que tenía el mock del front, con las mismas formas de respuesta.
src/seed.rs	Lista de colecciones y carga del seed (asigna passwordHash por defecto a los usuarios).
src/error.rs	ApiError { status, message } -> JSON { error }. Conversión desde rusqlite y serde_json.
src/util.rs	Lectura tolerante de JSON (s, f, b, arr, merge, set), fechas y números.
build.rs · .cargo/config.toml · lib/	Enlace con sqlite3.dll (SQLITE3_LIB_DIR) y copia de la DLL junto al ejecutable.

2.3 Decisiones de diseño (y por qué)

- **Documentos JSON en vez de tablas normalizadas.** El front ya tiene definidas las formas de cada entidad en su mock; guardarlas tal cual permitió conectar todas las pantallas sin tocar sus componentes. Para la versión final conviene normalizar (ver sección 8).
- **SQLite por DLL oficial, sin bundled.** La máquina no tiene compilador C; rusqlite se enlaza contra lib/sqlite3.dll y build.rs la copia al lado del binario. En otra máquina con gcc o MSVC puede usarse la feature bundled directamente.
- **JWT firmado a mano (HS256).** Las crates habituales de JWT dependen de código C/asm (ring). Con hmac + sha2 el binario compila en cualquier toolchain. Los claims son sub, email, role, iat, exp.
- **Mismas formas de respuesta que el mock.** Los endpoints de negocio devuelven exactamente lo que devolvía cada función del mock ({ data, receipt }, { success, participant }, etc.), así el front no distingue si está contra mock o API.
- **Un Mutex para la conexión.** Las operaciones son cortas y el tráfico es de prueba; en producción corresponde un pool y operaciones asincrónicas.

3. Modelo de datos

Tabla única **documents**: `collection TEXT, id TEXT, data TEXT (JSON), created_at, updated_at, PRIMARY KEY (collection, id)`. Las 15 colecciones y sus campos principales (los nombres son los que usa el front):

Colección	Prefijo id	Campos principales
usuarios	usr-	name, nombre, apellido, dni, phone, email (único), address, role (client | admin | coach), memberNumber, memberSince, activeDiscounts[], totalBookings, passwordHash (nunca se devuelve)
canchas	cancha-	name, surfaceType (nombre de tipos-cancha), capacity, pricePerHour, status (disponible | ocupada | mantenimiento), maintenanceNotes, description, photo, schedule[], availability { days[], openTime, closeTime, slotMinutes, blockedRanges[], notes }
tipos-cancha	ct-	name (único), label, description, speed, basePricePerHour, maintenanceFrequency, maxStudentsPerClass (<= 30), active
stock	stock-	name, category, description, totalStock, inUseStock, availableStock (calculado), minAlertThreshold, unit, status (normal | alerta_baja)
reservas	RES-AAAA-NNN	courtId, courtName, surfaceType, date, startTime, endTime, durationHours, playersType, participants[], equipmentAssigned, courtCost, equipmentCost, totalCost, depositPaid, depositReceiptNumber, depositPaymentMethod, remainingBalance, remainingPaid, finalReceiptNumber, status (confirmada | finalizada | cancelada), createdAt, cancellationDetails
torneos	trn-	name, competitionTypeId, category, modality, surfaceRequired, startDate, endDate, registrationFee, maxParticipants, currentEnrolled, status, description, internalRules, participants[], bracket[{ round, matches[] }]
ligas	liga-	igual que torneos + standings[{ rank, name, pj, pg, pp, sf, sc, dif, points }], fixtures[{ round, matches[] }]
tipos-competencia	cmp-	name, format, appliesTo (Torneo | Liga | Ambos), scoringSystem, minParticipants, maxParticipants, description, active
profesores	coach-	name, type (Profesor | Entrenador), dni, phone, email, specialty, experienceYears, hasOfficialLicense, availableHours, licenseDetails { titleName, institution, licenseNumber, issueDate, expiryDate, status, digitalDocumentUrl }, hourlyRatePrivate, photo, bio
clases	cls-	name, type, kind (clase | entrenamiento), coachId, coachName, scheduleDays, scheduleTime, courtAssigned, maxCapacity (<= 30), currentEnrolled, monthlyFee, monthName, sessions[], students[{ id, name, dni, phone, enrolledAt, attendance[] (P | A | J | -) }]
clases-particulares	prv-	kind, coachId, coachName, studentName, studentDni, studentPhone, date, time, courtAssigned, hourlyRate, status (programada | confirmada | cancelada), notes
recibos	REC-...	reservationId, clientName, clientDni, concept, items[{ description, amount }], totalPaid, paymentMethod, status, date, qrData, cashierName
facturas	FC-T-0001-NNNNNNNN	type (A | B | C), clientName, clientDni, clientAddress, concept, items[{ description, quantity, unitPrice }], subtotal, taxRate, taxAmount, total, paymentMethod, relatedReceiptId, status (emitida | anulada), date, issuedBy
reembolsos	RMB-AAAA-NNNN	clientName, clientDni, relatedReservationId, relatedReceiptId, reason, refundType (total | parcial), originalAmount, refundAmount, refundMethod, status (pendiente | procesado | rechazado), date, processedBy
descuentos	disc-	name, code (único, mayúsculas), discountPercent (0-100), applicableTo, active

4. Autenticación y seguridad

- **Contraseñas:** bcrypt (cost 4, suficiente para pruebas; usar 10-12 en producción). Se guardan en passwordHash y nunca se devuelven. Alta por POST /usuarios acepta password; si falta, se asigna 123456.
- **Token:** JWT HS256, claims { sub, email, role (Socio | Empleado | Profesor), iat, exp }, vigencia 8 horas, clave JWT_SECRET. El front guarda el token en localStorage y lo manda como Bearer (interceptor de axios).
- **Middleware:** todas las rutas bajo /api salvo health, auth/login y auth/registrar exigen un token válido; si falta o falla la firma o venció responde 401 { error }. Un token de Modo Demo del front (firma ficticia) es rechazado: por eso el front usa mocks en ese modo.
- **Autorización por rol:** no implementada. Cualquier usuario autenticado puede llamar cualquier endpoint (ver recomendaciones).
- **CORS:** permisivo (cualquier origen) para facilitar las pruebas desde otras PCs.

5. Referencia de endpoints

Base: `http://<host>:5090/api`. Cuerpos y respuestas en JSON. Códigos: 200 OK, 201 creado, 400 datos inválidos, 401 sin token o token inválido, 404 no existe, 409 conflicto de regla de negocio, 500 error interno.

5.1 Públicos

Método	Ruta	Cuerpo	Respuesta
GET	/health	—	{ status, service, version, sqlite, uptimeSeconds, collections }
POST	/auth/login	{ email, password }	AuthResponseDto: { token, id, nombre, apellido, email, numeroTelefono, direccion, rol, dni, memberNumber, expiraEn }
POST	/auth/registrar	{ nombre, apellido, email, password, direccion?, numeroTelefono?, dni? }	201 + AuthResponseDto (rol Socio). 409 si el email existe.

5.2 CRUD genérico

Método	Ruta	Descripción
GET	/{col}	Lista completa de la colección en orden de inserción (sin paginación).
GET	/{col}/{id}	Un documento (404 si no existe).
POST	/{col}	Alta. Genera id si falta y aplica defaults y validaciones de la colección (usuarios: email único; tipos-cancha: nombre único y cupo <= 30; stock: recalcula disponible y alerta; clases: cupo <= 30; descuentos: código único en mayúsculas; facturas: neto, IVA y total; reembolsos: total o parcial; reservas: delega en el alta de negocio). 201.
PUT	/{col}/{id}	Modificación por mezcla superficial. id inmutable. Reaplica validaciones (stock, tipos-cancha, clases, descuentos, usuarios con password). reservas recalcula fin, costo y saldo.
DELETE	/{col}/{id}	{ success: true }. tipos-cancha con canchas asociadas: 409. descuentos: se quita el código de todos los usuarios.

5.3 Negocio

Método	Ruta	Cuerpo	Respuesta / regla
GET	/auth/yo	—	Claims del token.
POST	/admin/reset	—	Vacía la base y recarga el seed: { success, documents }.
PATCH	/canchas/{id}/estado	{ status, maintenanceNotes }	Cancha. status en disponible | ocupada | mantenimiento.

Método	Ruta	Cuerpo	Respuesta / regla
PUT	/canchas/{id}/disponibilidad	{ days[], openTime, closeTime, slotMinutes, blockedRanges[], notes }	Cancha. 400 sin días o cierre <= apertura. (RF016-017)
PATCH	/stock/{id}/ajuste	{ totalStock }	Ítem con disponible y alerta recalculados.
POST	/reservas	reserva (courtId, courtName, date, startTime, durationHours, participants[], courtCost, totalCost, depositPaid, depositPaymentMethod...)	201 { data, receipt }. Crea el recibo de seña 50% (RD01). 409 si la cancha está en mantenimiento (RD06).
POST	/reservas/{id}/pagar-saldo	{ paymentMethod }	Reserva finalizada con finalReceiptNumber; crea recibo final. 409 si ya estaba liquidada o cancelada.
POST	/reservas/{id}/cancelar	{ hoursInAdvance }	{ data, isWithinFreeWindow, refund, penaltyReceipt }. >= 6 h: reembolso total pendiente (RD04). < 6 h: recibo de cargo por el 50% de la seña y reembolso parcial del resto (RD03). 409 si ya estaba cancelada.
PUT	/torneos/{id}/reglamento · /ligas/{id}/reglamento	{ internalRules }	Competencia actualizada (RF041 / RF048).
POST	/torneos/{id}/inscripciones · /ligas/{id}/inscripciones	{ name, dni, phone, partnerName, partnerDni }	201 { success, participant }. 409 si se alcanzó maxParticipants. En ligas agrega la fila en standings.
DELETE	/torneos/{id}/inscripciones/{pid} · /ligas/{id}/inscripciones/{pid}	—	{ success }. En ligas quita la fila de standings y reordena rank.
POST	/torneos/{id}/enfrentamientos · /ligas/{id}/enfrentamientos	{ round, p1, p2, court, date }	201 { success, match }. Crea la ronda o jornada si no existe. 400 si p1 = p2.
PUT	/torneos/{id}/enfrentamientos/{mid} · /ligas/{id}/enfrentamientos/{mid}	campos del partido (p1, p2, date, court, status, score, winner)	{ success, match }.
POST	/ligas/{id}/enfrentamientos/{mid}/resultado	{ score, winner }	{ success, match, standings }. Finaliza el partido y suma 3 puntos al ganador (una sola vez por partido). 400 si el ganador no es p1 ni p2.
PUT	/profesores/{id}/disponibilidad	{ availableHours }	Docente (RF104-105 / RF112-113).
PUT	/profesores/{id}/certificacion	campos de licenseDetails	Docente con licenseDetails mezclados (RF116-117 / RF121-122).
POST	/clases/{id}/alumnos	{ name, dni, phone }	201 { success, student } con attendance inicial. 409 si el cupo (maxCapacity <= 30) está completo (RD07 / RD10).
DELETE	/clases/{id}/alumnos/{sid}	—	{ success }.
POST	/clases/{id}/asistencia/alternar	{ studentId, sessionIndex }	{ success, classes } (todas las clases). Cicla P -> A -> J -> P.
POST	/clases/{id}/asistencia/sesion	{ sessionIndex, status }	{ success, classes }. Marca a todos los alumnos.
POST	/clases/{id}/asistencia/registrar	{ studentId, status }	{ success }. Agrega una sesión al alumno.
POST	/clases-particulares/{id}/cancelar	—	{ success, data } con status cancelada.
POST	/facturas/{id}/anular	—	Factura con status anulada.
PATCH	/reembolsos/{id}/estado	{ status }	Reembolso. status en pendiente | procesado | rechazado.
PUT	/usuarios/{id}/descuentos	{ activeDiscounts[] }	Usuario. 400 si algún código no existe en descuentos.

6. Reglas de dominio implementadas en la API

Regla	Dónde	Comportamiento
RD01 — No se confirman reservas pendientes de pago	POST /reservas	El alta registra la seña del 50% y emite el recibo; la reserva nace confirmada con el saldo pendiente.
RD02 / RD04 — Cancelación con antelación mínima y reembolso	POST /reservas/{id}/cancelar	Con 6 h o más se genera un reembolso total (pendiente de proceso en Caja).
RD03 — Cargo por cancelación fuera de plazo	POST /reservas/{id}/cancelar	Con menos de 6 h se emite un recibo de cargo por el 50% de la seña y un reembolso parcial por el resto.
RD06 — Cancha bloqueada por mantenimiento	POST /reservas, PUT /reservas/{id}	409 si la cancha elegida está en mantenimiento.
RD07 — Cupo máximo de 30 alumnos	POST/PUT /clases, POST /tipos-cancha	400 si maxCapacity > 30; maxStudentsPerClass se recorta a 30.
RD10 — Cupo configurable por comisión	POST /clases/{id}/alumnos	409 cuando la clase alcanza su propio maxCapacity.
RD11 — Medios de cobro	recibos / facturas	paymentMethod es texto libre; el front ofrece débito, crédito, billetera virtual y efectivo.
RD12 — Reglamento interno por competencia	PUT /{torneos|ligas}/{id}/reglamento	internalRules por torneo y por liga; tipos-competencia describe el formato oficial.
RD05 — Antelación máxima de reserva	—	No validada en la API (el front no la envía). Pendiente para la versión final.
RD08 / RD09 — Título habilitante y certificación	profesores.licenseDetails	Se almacena y se expone; no se bloquea la asignación de clases a docentes sin certificación vigente.

7. Integración con el front

- **src/services/httpApi.js**: implementación HTTP de los 13 servicios (courts, courtTypes, stock, reservations, tournaments, competitionTypes, leagues, coachesAndClasses, receipts, discounts, invoices, refunds, users) con la misma interfaz que el mock. Los errores { error } se lanzan como Error(message).
- **src/services/api.js**: los mocks pasaron a ser privados (mockXxxService) y cada export es withBackend(mock, http). La decisión se toma por llamada: HTTP si GET /api/health respondió ok en los últimos 20 s y el usuario guardado no es Modo Demo.
- **Modo Demo** (botones Demo Socio / Demo Admin / Demo Docente) sigue funcionando sin backend, con datos en memoria. Con la API levantada, iniciar sesión con admin@tenisahora.com / 123456 para trabajar con datos persistidos.
- **AuthContext**: mapea rol Socio -> client, Empleado -> admin, Profesor -> coach; toma dni y memberNumber del AuthResponseDto si vienen.
- **Build servido por la API**: npm run build con VITE_API_URL=/api genera dist/ con la API relativa; la API lo sirve en / con fallback a index.html, por lo que un solo puerto alcanza para front y API (sin CORS entre orígenes).
- **Servidor de desarrollo**: npm run dev sigue usando http://localhost:5090/api por defecto; vite.config.js admite hosts *.nip.io para abrirlo desde otra PC.

8. Ejecución y despliegue de prueba

8.1 En la máquina de desarrollo

```
cd D:\fede\api
.\run.ps1           # cargo build + ejecuta en http://localhost:5090/api
python tests\smoke.py  # prueba de humo contra la API corriendo

cd D:\fede
```

```
.\start-servidor.ps1  # build release de la API + build del front; deja la API corriendo oculta
.\stop-servidor.ps1   # detiene la API
```

8.2 Acceso desde otra PC (nip.io)

nip.io resuelve cualquier nombre de la forma `<ip>.nip.io` a esa IP, así se puede usar un hostname sin configurar DNS. La API escucha en 0.0.0.0:5090 y sirve el front en la raíz. URLs detectadas al dejar el servidor corriendo:

URL	Interfaz
http://192.168.100.69.nip.io:5090/ (o http://192.168.100.69:5090/)	Ethernet 4 (DHCP)
http://192.168.1.15.nip.io:5090/ (o http://192.168.1.15:5090/)	Ethernet 3 (IP fija)
http://localhost:5090/	misma PC

La PC remota tiene que estar en la misma red (o tener ruta a esa IP) y resolver DNS público para nip.io. El firewall de Windows tiene regla de entrada "Tenis Ahora API" para el puerto 5090. La IP pública 84.233.234.201 corresponde a la VPN activa en la máquina, no a un acceso entrante.

8.3 Variables de entorno

Variable	Default	Uso
PORT	5090	Puerto HTTP.
DATABASE_PATH	data/tenis_ahora.sqlite	Archivo SQLite (se crea solo).
SEED_PATH	seed/seed.json	Datos iniciales, cargados solo si la base está vacía.
JWT_SECRET	valor de desarrollo	Clave HS256.
FRONTEND_DIST	../Tenis-Ahora-Frontend/dist	Carpeta del build del front a servir (opcional).
RUST_LOG	info,tower_http=info	Nivel de log.

9. Verificación realizada

9.1 Prueba de humo (tests/smoke.py)

Resultado: **45 de 45 verificaciones OK**. Cubre:

- Health, rutas protegidas sin token (401), login incorrecto (401), login admin, /auth/yo, registro y registro duplicado (409).
- Reset y seed; CRUD de tipos de cancha con recorte de cupo, duplicado (409), modificación, baja en uso (409), baja y 404.
- Estado y disponibilidad de canchas; reserva en cancha bloqueada (409, RD06); disponibilidad sin días (400).
- Reservas: alta con recibo de seña, modificación con recálculo de fin, costo y saldo, pago de saldo, cancelación dentro y fuera de plazo (reembolso total, cargo + reembolso parcial), doble cancelación (409), baja.
- Torneos: inscripción, enfrentamiento con resultado, jugador repetido (400), reglamento. Ligas: resultado suma 3 puntos y no duplica al recargar.
- Clases: cupo propio (409, RD10), cupo > 30 (400, RD07), alternar asistencia, marcar sesión completa.
- Caja: factura con IVA y anulación, reembolso parcial y procesamiento, descuento con código en mayúsculas, alta de usuario sin exponer hash, asignación de descuentos, código inexistente (400), cascada al borrar descuento, login del usuario creado por admin.

9.2 Front contra la API (navegador)

- Login con admin@tenisahora.com / 123456 contra la API: sesión sin Modo Demo (isDemo=false), rol admin, número de socio ADM-001 tomado del AuthResponseDto.
- Alta de un tipo de cancha desde la pantalla de Canchas y recarga completa de la página: el registro persiste (antes, con mocks, desaparecía).
- Cancelación de una reserva con 3 hs de antelación: el aviso informa el cargo REC-CARGO generado; en Caja aparecen el reembolso parcial y el recibo de cargo, ambos leídos de la API.
- Pantallas de Docentes (matriz de asistencia) y Usuarios cargando desde la API; 31 llamadas de datos a /api durante el recorrido, sin errores de consola.
- Rutas profundas de la SPA (por ejemplo /admin/torneos) responden 200 con index.html desde la API.
- Modo Demo (Demo Socio / Demo Admin) sigue funcionando con datos en memoria, independiente de la API.
- Acceso por http://192.168.100.69.nip.io:5090/: nombre resuelto por DNS, front y /api/health responden, y la prueba de humo completa (45/45) pasa a través de ese hostname.

10. Limitaciones conocidas

- Sin autorización por rol: un Socio autenticado puede llamar endpoints administrativos.
- Sin paginación ni filtros en los GET de colección; se devuelve todo.
- Validación de tipos tolerante: los campos se leen con defaults; un valor con tipo incorrecto no produce 400 sino que se ignora o se convierte.
- Documentos JSON sin integridad referencial (una reserva puede referir una cancha borrada).
- Un solo Mutex sobre la conexión SQLite: correcto pero sin concurrencia real.
- Tokens sin revocación ni refresh; CORS abierto; HTTP sin TLS.
- Los recibos de seña y los reembolsos se generan con ids pseudoaleatorios no secuenciales (mismo criterio que el mock).
- RD05 (antelación máxima de reserva) y la verificación de certificación vigente al asignar docentes no están implementadas.

11. Recomendaciones para la versión definitiva

Tema	Recomendación
Modelo de datos	Normalizar en tablas con claves foráneas (usuarios, canchas, tipos_cancha, reservas, recibos, facturas, reembolsos, torneos, ligas, participantes, enfrentamientos, profesores, certificaciones, clases, inscripciones, asistencias, descuentos, usuario_descuento). Mantener los nombres de campo del contrato JSON en la capa de salida (DTOs) para no romper el front.
Contrato HTTP	Conservar rutas, cuerpos y formas de respuesta de las secciones 5 y 6: el front ya está escrito contra ellas. Si se cambian, actualizar src/services/httpApi.js. Publicar una especificación OpenAPI.
Autenticación	JWT con biblioteca estándar, expiración corta + refresh token, revocación, bcrypt/argon2 con costo de producción, rate limiting en login.
Autorización	Roles por endpoint: Empleado para altas/bajas administrativas, Socio para sus reservas e inscripciones, Profesor para asistencia de sus clases.
Validación	Tipos fuertes (serde structs con validación) y respuestas 400 con detalle de campo. Validar fechas y solapamiento de turnos contra availability y reservas existentes.
Reglas pendientes	RD05 (antelación máxima configurable), RD08/RD09 (bloquear docentes sin certificación vigente), numeración fiscal real de facturas y recibos, política de reembolsos parametrizable.
Persistencia	Pool de conexiones (sqlx o diesel con migraciones) o motor cliente-servidor (PostgreSQL). Transacciones en las operaciones compuestas (reserva + recibo, cancelación + reembolso + cargo).
Operación	HTTPS, CORS restringido al origen del front, logs estructurados, health con chequeo de base, variables de entorno obligatorias en producción, contenedor o servicio del sistema.
Calidad	Tests de integración por endpoint (hoy: smoke.py), CI con cargo test y cargo clippy, seed solo en entornos de prueba.